Requirements Engineering (2026) 31:7
https://doi.org/10.1007/s00766-026-00462-z

ORIGINAL ARTICLE

From issue titles to requirements: an empirical study of large language models and prompt engineering strategies

Guilherme Paiva[1] · **Edna Dias Canedo**[1] · **Geraldo Pereira Rocha Filho**[2]

Received: 21 June 2025 / Accepted: 21 April 2026
© The Author(s) 2026

Abstract

This study empirically assesses how effectively two Large Language Models (LLMs), `OpenAI o3-mini` and `DeepSeek R1`, can transform terse feature-request titles from open-source software (OSS) issue trackers into well-formed software requirements. It further examines how prompt-engineering strategies shape requirement quality and evaluates a scalable "LLM-as-a-Judge" approach for automated quality assessment based on three ISO/IEC/IEEE 29148:2018 quality attributes: *Unambiguity*, *Verifiability*, and *Singularity*. We extract 150 feature-request titles from five OSS repositories and pair each title with every combination of two LLMs and three prompt styles, producing 900 candidate requirements. An independent evaluator LLM rates each requirement on these three quality attributes using a Likert scale and provides textual rationales. Ordinal data are analyzed with descriptive statistics and non-parametric tests, complemented by thematic analysis of the evaluator's explanations. A targeted human validation study with five evaluators on a stratified sample of 50 requirements assesses the reliability of the automated judge. LLMs often produce high-quality requirements; however, scores vary with input clarity and prompt design. Few-shot prompting consistently boosts *Singularity*, while Expert Identity prompting sometimes raises *Verifiability* but frequently harms *Singularity*. `o3-mini` shows a modest but significant edge in *Singularity*. Qualitative review echoes these patterns, revealing trade-offs between added detail and focus. The LLM-as-a-Judge protocol delivers consistent, scalable evaluations whose scores correlate significantly with aggregate human judgment ($p < 0.05$ for all three attributes). Modern LLMs can expedite the drafting of initial software requirements from informal OSS inputs, but their output quality hinges on careful prompt selection and the inherent clarity of the source title. Prompt effects are model-dependent and may introduce trade-offs among quality attributes, so human oversight remains indispensable for refinement. The LLM-as-a-Judge framework proves a practical, human-validated technique for large-scale evaluation, enabling rapid, reproducible insights into LLM-driven requirements engineering workflows.

Keywords Requirements engineering · Large language models · Prompt engineering · Automated quality assessment · Open-source software issues · Requirements statements

1 Introduction

Large Language Models (LLMs) have gained increasing importance in artificial intelligence (AI) research because of their sophisticated capabilities in handling complex tasks and generating high-quality natural language text. In the field of Natural Language Processing (NLP), these models have redefined how algorithms comprehend and generate language, revolutionizing applications in a variety of domains [23, 54]. Requirements Engineering (RE) plays a central role in the software development lifecycle by defining system objectives, functions, and constraints, directly influencing product quality and project success [18, 24, 36, 39, 45]. Recent studies also emphasize that RE practices and

✉ Guilherme Paiva
guipaivagpp@gmail.com

✉ Edna Dias Canedo
ednacanedo@unb.br

Geraldo Pereira Rocha Filho
geraldo.rocha@uesb.edu.br

[1] Department of Computer Science, University of Brasília, Brasília, Brazil

[2] Southwest Bahia State University, Vitória da Conquista, Brazil

Published online: 26 May 2026

tools are fundamental for addressing challenges in modern contexts such as AI and ML-enabled systems [3, 46]. LLMs are therefore particularly promising for RE tasks, where the efficiency and precision of requirements elicitation, specification, and documentation have a profound impact on project success [26].

By automating traditionally manual and error-prone tasks, LLMs can mitigate challenges such as conflicting requirements, time constraints, and limited expertise among stakeholders and developers [5]. However, rather than aiming for full automation, current evidence suggests that LLMs are most effective as assistive partners that augment human capabilities in RE. They can accelerate repetitive and low-level tasks while keeping analysts in the loop for interpretation, validation, and negotiation. This perspective frames LLMs not as replacements for human expertise, but as complementary tools that enhance productivity and consistency in RE activities.

Although the adoption of AI in RE has grown, significant challenges remain, particularly in automating requirements generation and management for Open-Source Software (OSS) projects. Current AI-based RE tools primarily address the identification and classification of requirements [33], relying mainly on traditional NLP methods rather than generative models [11]. These approaches frequently fall short in domains requiring deeper automation, such as requirements specification and elicitation. Moreover, OSS environments often gather requirements through diverse and indirect feedback channels, making the extraction process labor intensive and inconsistent [44]. Most of these technologies also rely heavily on supervised learning that requires extensive labeled data [1], creating scalability challenges for OSS projects where such datasets are scarce. Recent pipelines for automating requirements generation [28, 31, 32] (discussed in Sect. 2.1) advance the state of the art but rely on richer input artifacts, leaving the challenge of generating quality-assessed requirements from concise OSS issue titles unaddressed. Approaches such as zero-shot and few-shot learning, which can operate with minimal or no labeled data, are therefore gaining traction as scalable alternatives [33, 42].

Although LLMs have shown abilities in several RE related tasks, their role in OSS settings remains underexplored. Prompt engineering, for example, has emerged as a promising strategy for improving LLM performance [35, 50], yet its effectiveness for requirements generation under minimal contextual information is not fully understood.

This study addresses these gaps by empirically evaluating the effectiveness of two distinct LLMs, `OpenAI o3-mini`[1] and `DeepSeek R1`,[2] in automatically generating software requirements directly from feature request issue titles sourced from active OSS repositories. We systematically investigate the impact of three prompt engineering strategies such as Naive zero-shot, Few-shot, and Expert Identity on the quality of the generated requirements.

The quality is assessed using key characteristics derived from the ISO/IEC/IEEE 29148:2018 [25] standard: Unambiguity, Verifiability, and Singularity via a scalable LLM-as-a-Judge methodology. Our aim is to understand how the choice of LLM and prompting strategy interact and influence requirement quality under minimal context conditions.

The findings of this research provide valuable empirical evidence regarding the practical application of LLMs in RE. Our results highlight that while LLMs show considerable potential, their effectiveness is modulated by prompt design and input quality. Certain prompts, such as Few-shot, consistently improve Singularity, whereas others, like Expert Identity, introduce model dependent trade-offs. These insights can guide practitioners in selecting appropriate prompting strategies while emphasizing the continued necessity of human oversight.

Furthermore, evaluating LLM generated requirements presents its own challenges. Traditional human evaluation is time consuming and costly, and ensuring consistency across multiple raters is difficult. Recent work has explored the "LLM-as-a-Judge" paradigm as a way to achieve scalable, consistent evaluation [55], although it carries potential limitations such as model bias [20]. In this study, we operationalize LLM-as-a-Judge to assess intrinsic requirement quality under different prompting conditions.

In this study, requirement quality is understood as the intrinsic linguistic quality of an individual requirement statement, rather than its project level adequacy or completeness. Following ISO/IEC/IEEE 29148 and recent research [18, 28, 36], we focus on three intrinsic attributes Unambiguity, Verifiability, and Singularity which can be assessed independently of domain context.

In order to achieve the objective of this research, we answer the following research questions:

RQ.1: How effective are LLMs in generating software requirements from feature requests in open source software projects?

RQ.2: To what extent does the prompt engineering strategy influence the quality of requirements generated by LLMs?

RQ.3: What are the common qualitative characteristics, strengths, and weaknesses observed in the requirements generated by the different LLMs and prompt styles?

[1] https://openai.com/index/openai-o3-mini/

[2] https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-70B

The main contributions of this research are summarized as follows:

C1: An empirical evaluation of two distinct LLMs, `o3-mini` and `DeepSeek R1`, for generating software requirements from OSS GitHub issue titles.

C2: A systematic comparison of three prompt engineering techniques and their influence on Unambiguity, Verifiability, and Singularity.

C3: Findings demonstrating that Few-shot prompting consistently improves Singularity, while Expert Identity prompting introduces model dependent trade-offs.

C4: Validation of the LLM-as-a-Judge approach for scalable evaluation of generated requirements.

C5: Qualitative insights into strengths (e.g., capturing core intent) and weaknesses (e.g., dependency on input clarity) of LLM based requirement generation.

2 Background and related work

The ISO/IEC/IEEE 29148:2018 [25] standard defines requirements elicitation as a systematic process to identify and document customer and end-user needs using structured techniques such as prototyping and surveys. This phase is crucial for translating often ambiguous stakeholder needs into clear, actionable requirements that align with the project's goals. Elicitation integrates diverse perspectives from stakeholders, including end users, acquirers, regulatory bodies, and development teams, each providing unique insights and priorities. The success of this phase depends primarily on the expertise of stakeholders and the effectiveness of communication among them, while the complexity of the system acts as an influencing factor that typically requires more structured and rigorous RE practices rather than determining success or failure directly.

Building on the elicitation phase, the specification phase refines these insights into formal requirements documents, following ISO/IEC/IEEE 29148:2018 guidelines [25]. It focuses on structuring requirements to ensure that they are clear, feasible, and verifiable, transforming stakeholder needs into categorized requirements business, stakeholder, system, and software specifications aligned with the development lifecycle. Specifications should be complete, correct, and consistent, incorporating verification and validation to confirm alignment with stakeholder expectations and feasibility within technical and organizational limits [25]. However, elicitation and specification face several challenges, including conflicting requirements, tight timelines, and difficulties in engaging stakeholders who may have limited availability or understanding of the process [5, 40]. The pervasive use of natural language in requirements often leads to ambiguity, inconsistency, and incompleteness, further complicating interpretation and precision [17].

The ISO/IEC/IEEE 29148:2018 [25] standard has also been widely adopted to define requirement quality characteristics such as Unambiguity, Verifiability, and Singularity. Nevertheless, recent studies have noted that these definitions can be simultaneously too generic and too restrictive [16, 18]. Our rationale for adopting this framework is threefold. First, ISO 29148 remains the most recognized reference for requirement quality and provides conceptual comparability with prior research in both academia and industry. Second, its linguistic focus allows assessing textual clarity and testability without access to domain-specific context, which is important when analyzing OSS issue titles. Third, it offers a neutral and interpretable foundation for LLM-as-a-Judge evaluation, where models act as linguistic assessors rather than domain experts. While other approaches, such as the template based benchmarking proposed by Großer et al. [19], operationalize requirement quality into quantifiable metrics, they rely on predefined templates or annotated datasets that are unavailable in our OSS based setting. Therefore, using ISO based intrinsic quality attributes represents a theoretically consistent and empirically feasible choice for assessing the linguistic quality of automatically generated requirements.

To address the aforementioned complexities, recent advancements in generative artificial intelligence, particularly LLMs, offer promising solutions to augment the requirements engineering process. LLMs excel at understanding and processing natural language, making them well-suited to transform informal stakeholder inputs into structured and precise requirement statements, thereby tackling the very issues of ambiguity and inconsistency that plague traditional methods [4, 17]. This capability can significantly reduce the manual effort and potential for human error in tasks such as requirements elicitation and validation [38].

To illustrate the feasibility of this transformation, consider the following issue title from the PyTorch repository: *"Implement* `numpy.random.choice` *equivalent."* When processed by an LLM with a simple (naive) prompt, the model produces the following requirement: *"Implement a function in PyTorch's random module that replicates the behavior of numpy.random.choice, supporting selection from a 1-D tensor or range of integers with options for sampling with or without replacement, probability weights, and multidimensional output."* This example demonstrates that even a terse, developer-oriented title can serve as a seed from which an LLM generates a structured requirement statement with clearly defined scope and behavior–attributes that are assessable against ISO/IEC/IEEE 29148 quality criteria.

By leveraging techniques like prompt engineering and zero-shot learning [1, 42], organizations can harness LLMs to improve consistency, resolve ambiguities early, and ultimately streamline the entire RE workflow. Recent empirical work also demonstrates that LLMs can assess requirement statements according to ISO 29148 characteristics such as Unambiguity, Verifiability, and Singularity, achieving partial agreement with human evaluators [28, 36].

2.1 Related work

In recent years, the field of RE has increasingly employed generative AI, particularly LLMs, to address the complexity and manual effort traditionally associated with requirements elicitation, analysis, and documentation. One of the primary motivations for using LLMs in RE is their potential to automate labor-intensive tasks, thereby enhancing efficiency and consistency. For example, Arora et al. [4] discuss the potential of LLMs, such as OpenAI's GPT series[3] and Google's BERT [12], in addressing early-stage ambiguities and enhancing the accuracy of requirements documentation. The authors posit that LLMs can be utilized throughout the RE process, from elicitation to validation, thus facilitating the generation of clear, unambiguous requirements in natural language, which is essential for both communication and automation in RE workflows.

Similarly, Marques et al. [38] provide a comprehensive analysis of ChatGPT's impact on RE, particularly with regard to its effectiveness in requirements elicitation and validation. The authors underscore that LLM technology can facilitate communication between stakeholders and enhance the precision of requirements by accurately capturing user needs. However, the study also highlights the challenges and ethical considerations associated with the use of LLMs in sensitive domains such as software development, particularly in terms of ensuring data privacy and addressing potential biases in model outputs.

One novel approach to RE automation is the integration of zero-shot learning (ZSL) techniques, as investigated by Alhoshan et al. [1]. This approach addresses the challenge of limited labeled data in RE by enabling LLMs to classify requirements without extensive training data. ZSL has demonstrated promising results in categorizing requirements (e.g., functional vs. non-functional), which is particularly beneficial in contexts where labeled datasets are scarce or costly to generate.

The use of collaborative multi agent frameworks, such as the MARE framework proposed by Jin et al. [26], further extends the application of LLMs in RE by enabling structured interaction among the various roles involved in the RE process. MARE divides the RE workflow into four stages: elicitation, modeling, verification, and specification, and assigns specific tasks to different AI agents. This approach improves both the quality of the generated requirements models and the overall accuracy and completeness of the resulting specifications through agent collaboration and shared workspaces.

Prompt engineering has emerged as a critical technique for optimizing LLM performance in RE, as evidenced by studies such as those conducted by White et al. [50] and Ronanki et al. [42]. White et al. [50] presented a catalog of prompt patterns that enhance LLM responses for specific RE tasks, including requirement elicitation and code generation. These patterns facilitate consistent interactions with LLMs, ensuring that the generated content aligns with RE goals. Similarly, Ronanki et al. [42] investigated how prompt patterns can be optimized for requirements classification and traceability, concluding that structured prompt engineering can improve the accuracy and relevance of LLM output, making them viable for RE automation.

In their study, Lim et al. [33] concentrate on the use of data-driven methodologies for requirements elicitation, highlighting the potential of employing both static and dynamic data sources. While traditional elicitation relies on direct stakeholder input, data-driven approaches harness diverse data sources, such as user feedback from digital platforms. This approach, while promising, still requires advanced techniques to handle the scale and variety of modern data, an area where LLMs could potentially make a significant contribution by automating data extraction and interpretation to generate actionable requirements.

More recently, advances in requirements generation have explored LLM-based and pipeline based automation. Lian et al. [32] propose a vision-to-specification pipeline that transforms conceptual feature descriptions into structured functional requirements, while Lian et al. [31] incorporate formal verification standards to automatically derive security requirements from functional specifications, reinforcing the importance of verifiable and standards-compliant artifacts. Krishna et al. [28] empirically evaluate the use of LLMs for producing software requirements specifications, identifying both their potential and common pitfalls such as verbosity and degradation of structure. These studies collectively advance automation in requirements generation and evaluation, but they rely on richer input artifacts (e.g., conceptual descriptions, functional specifications, or SRS fragments) than the concise and noisy OSS issue titles considered in our work.

While the reviewed literature establishes a clear trajectory towards leveraging LLMs in RE, it also reveals specific research gaps that this study aims to address. Foundational work on prompt engineering in RE by White et al. [50] and

[3] https://platform.openai.com/docs/models

Ronanki et al. [42] has successfully identified and cataloged effective prompt patterns. However, there remains a need for large-scale, comparative empirical studies that systematically measure how distinct strategies (e.g., few-shot examples versus expert personas) influence specific, standard-defined quality attributes like *Unambiguity*, *Verifiability*, and *Singularity*. Although recent work has begun to explore LLM-based requirements generation from richer inputs such as conceptual descriptions [31, 32] and SRS fragments [28], much of the earlier literature concentrated on adjacent tasks such as requirement classification [1] or higher-level collaborative frameworks [26]. None of these studies addresses the fundamental challenge of generating a single, well-formed requirement from a terse, informal input such as an OSS issue title. In addition, the interactive effect between the choice of LLM and the prompting strategy remains underexplored. Our work directly addresses these gaps by conducting a systematic empirical evaluation that not only compares prompt techniques but also examines their model-dependent trade-offs on requirement quality. By doing so, we extend the literature beyond proposing patterns to providing documented evidence of their performance and side effects, offering more nuanced guidance for their practical application in RE.

3 Research method

This study investigates the use of LLMs in software requirements engineering, assessing the effectiveness of prompt-engineering techniques to generate requirements from feature requests extracted from open-source software repositories. The methodology is divided into three main stages: data collection, requirements generation, and requirements evaluation as in Fig. 1. Each stage was carried out systematically to ensure the validity, replicability, and reliability of the results.

3.1 Data extraction

As illustrated in Fig. 1 – step 1, to establish a comprehensive dataset of requirements-relevant data, we extracted feature requests from the top five most active open source projects on GitHub by number of issues: **PyTorch**[4] (machine learning framework), **Flutter**[5] (UI framework for mobile and web development), **Godot Engine**[6] (game development engine), **Rust**[7] (systems programming language), and **Go**[8] (backend and network programming language).[9] These repositories were chosen based on their high level of activity and relevance to the open source community. This selection intentionally spans multiple software domains to capture linguistic and contextual variability across issue titles and ensure broader representativeness of OSS requirements. The following steps were taken in data collection:

1. *Issue Selection Criteria:* We selected the 30 most active[10] open issues from each repository to ensure the inclusion of genuine feature requests. This approach was necessary because there is no consistent programmatic method to identify issues that were closed due to misclassification, spam, or duplication across repositories. The issues were filtered to include only those labeled as "feature request". This filtering was essential to ensure that the data set focused on actionable, requirement-specific content, excluding irrelevant discussions or minor tasks. Selecting the thirty most active feature requests per repository, ranked by the number of reactions, ensured that the dataset captured issues representing genuine

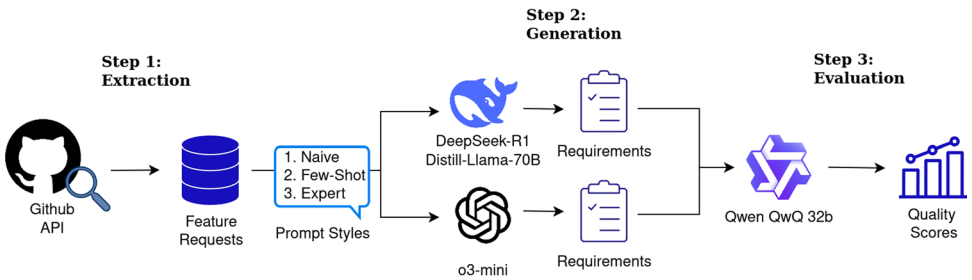


Fig. 1 Overall methodological workflow of the study. Step 1: Feature requests are extracted from GitHub repositories. Step 2: Two LLMs generate requirements using three distinct prompt-engineering strategies. Step 3: The generated requirements are evaluated by Qwen QwQ-32b based on ISO 29148 attributes, producing quality scores

[4] https://github.com/pytorch/pytorch

[5] https://github.com/flutter/flutter

[6] Godot Engine has a specific repository for feature requests: https://github.com/godotengine/godot-proposals

[7] https://github.com/rust-lang/rust

[8] https://github.com/golang/go

[9] Data collected on Mar 22, 2025 from the repository https://github.com/EvanLi/Github-Ranking

[10] Sorted by reactions on GitHub's API: https://docs.github.com/en/rest/reactions/reactions?apiVersion=2022-11-28

and widely discussed functional needs while filtering out duplicates, spam, or minimally descriptive titles. Although less active issues may vary in style or completeness, exploratory inspection revealed that they often lacked clear requirement intent, making them unsuitable for systematic comparison. This filtering strategy thus prioritizes input quality and reproducibility, enabling fair evaluation of LLM behavior across well-structured and semantically rich issue titles.

2. *Data Extraction and Formatting:* Using GitHub's API, issue titles and labels were extracted and structured into a standardized dataset. Each entry contained metadata (e.g., repository name, issue ID) alongside the textual content necessary for requirement generation. This step facilitated consistency across the dataset, helping prompt engineering and later stages of analysis, while also enabling reproducibility of the experiments.

Thus, the dataset comprised 150 feature requests collected equally from the five distinct repositories previously mentioned. For the subsequent requirements generation phase, the 'Issue Title' extracted from each request served as the primary input data. This curated collection presents a diverse range of inputs for the LLMs, encompassing requests from varied software domains (such as machine learning, UI frameworks, game development, and systems programming) and exhibiting natural variations in phrasing, technical specificity, and title length. These variations are also influenced by the domain-specific terminology of each project, for example, Pytorch frequently references model training and tensor operations, while Flutter emphasizes widgets and UI components, and Godot includes terms from game mechanics. Such diversity in technical vocabulary reflects the natural heterogeneity of OSS requirements and was intentionally preserved to ensure linguistic representativeness across different software domains.

By restricting the input to issue titles only, we deliberately model a minimal context scenario that is common in OSS backlogs and focus our analysis on how well LLMs can transform such terse, informal signals into linguistically well formed requirement statements, rather than on assessing project level adequacy, completeness, or stakeholder alignment.

An excerpt illustrating the structure of the collected data is shown in Table 1.

Table 1 Excerpt of the feature request dataset

Repository	Issue Number	Issue Title
pytorch/pytorch	16,897	Implement `numpy.random.choice` equivalent
flutter/flutter	46,789	Improve the indexability (SEO) of Flutter apps on the web
godotengine/godot-proposals	6416	Add a Trait system for `GDScript`
rust-lang/rust	39,915	Linking with LLD
golang/go	32,204	`net/http`: support HTTP/3

3.2 Requirements generation

In the second step of our methodology, we prepared the prompts and executed two different LLMs to generate requirements from the dataset built in the first step. Motivated by previous studies on prompt engineering [13, 34, 50], we built our prompts based on some of the most known techniques to assess their role on improving the quality of LLM responses.

3.2.1 Prompts setup

We developed three distinct prompts based on known prompt engineering techniques to investigate how they affect the quality and clarity of the requirements produced: Naive, Few-shot, and Expert Identity. Each method represents a different strategy in how the model is primed to interpret and respond to the input.

Naive This method involved directly feeding the issue titles to the model without further details or examples. Based on zero-shot learning [47], this strategy relies on general knowledge of the model. As explored in the study by Alhoshan et al. [1] this technique showed its effectiveness in requirements classification tasks without labeled training data. In this work, we are using it to test the model's inherent knowledge and adaptability without context.

Prompt - Naive

You are given a feature request from an open-source software project's issue tracker. Generate a clear, concise, and self-contained software requirement statement that captures the intended feature. Output only the finalized requirement statement. Do not include explanations or commentary.

Few-shot Here, a few examples of previously generated requirements were included with the prompt to guide the model. This technique helps the model to infer the task requirements from minimal examples, enhancing its accuracy and relevance [6, 13]. This prompt technique aimed to give the model minimal guidance for better accuracy based on patterns from past examples.

Prompt - Few-Shot

Here are examples of feature requests from an open-source software project's issue tracker along with their corresponding software requirements.

Based on these examples, analyze new feature requests and generate the requirement.

Output only the finalized requirement statement. Do not include explanations or commentary.

Feature request: Limit GPU memory usage during prediction.

Requirement: The system shall provide an option to limit GPU memory usage during prediction.

Feature request: Reset password

Requirement: The system must provide a functional "forgot password" link that allows users to reset their passwords securely.

Feature request: Make the application use vector search.

Requirement: The system shall use vector search to improve search performance.

Expert Identity Expert Prompting instructs LLMs to act as distinct experts to solve complex problems, providing an expert identity and instructions specific to their domain. This leads to better performance and higher quality answers, especially in tasks that require a thorough understanding [52].

Prompt - Expert

You are a Software Requirements Engineer with deep expertise in analyzing feature requests in open-source software projects. Your task is to generate a clear, complete, and actionable software requirement from informal feature requests found in OSS issue trackers.

Output only the finalized requirement statement. Do not include explanations or commentary.

3.2.2 Prompts execution

The generation of requirements was performed using the `OpenAI o3-mini`[11] and the `DeepSeek-R1-Distill-Llama-70B`[12] [10] reasoning models. The models were selected based on their average performance across multiple benchmarks. We chose `OpenAI o3-mini` for its superior cost-effectiveness among closed-source reasoning models: it costs less than 10% of `OpenAI o1`, the top-performing reasoning model, while delivering comparable results. Similarly, we selected DeepSeek-R1-Distill-Llama-70B as the most cost-effective open-source reasoning model, based on the model size of its distilled variants and benchmark performance metrics[13] [49].

To ensure a degree of reproducibility while fostering output diversity, we configured the LLMs with specific parameters, fixing the random seed to 42 for all generation tasks. Each configuration included standard decoding parameters such as temperature, which controls the degree of randomness in token sampling. We employed a temperature setting of 1.0; while lower values yield more deterministic outputs, higher values promote lexical diversity by scaling the logits in the softmax layer. This choice is supported by research on neural text degeneration, which demonstrates that maintaining sufficient entropy through stochastic sampling is essential to prevent the model from falling into repetitive loops often observed in deterministic decoding [22]. Although LLM responses can exhibit non-deterministic behavior even with a fixed seed due to underlying model optimizations, repeated executions within the scope of this study produced highly consistent outputs. Randomly sampled comparisons across multiple runs revealed negligible variation, thereby demonstrating the practical reproducibility of our experimental results.

We distinguished between system and user prompts following established practices in LLM-based evaluation studies [21, 36]: the former defines the model's role or global behavior (e.g., "You are an expert requirements analyst"), while the latter provides task-specific input for each generation. We evaluated both LLMs by utilizing all prompt variants as *system prompts* and issue titles as *user prompts*. Each prompt variant was paired with every issue title and submitted to both models, resulting in three distinct requirements per issue. This process yielded a total of 450 requirements generated by each LLM, amounting to a dataset of 900 requirements overall.

3.3 Requirements evaluation

3.3.1 Evaluation metrics

In the realm of systems and software engineering, well-formed requirements are crucial for successful project outcomes. According to ISO/IEC/IEEE 29148:2018 [25], individual requirements should be: 1. necessary; 2. appropriate; 3. unambiguous; 4. complete; 5. singular; 6. feasible; 7. verifiable; 8. correct; and 9. conforming. A closer analysis of these characteristics leads to the conclusion that **unambiguity**, **singularity**, and **verifiability** can be easily assessed when evaluating requirements in isolation. In this study, we therefore focus exclusively on these three attributes, as they constitute *intrinsic* requirement quality and can be meaningfully assessed without domain or stakeholder context. This focus aligns with recent RE research arguing that intrinsic linguistic properties are the only attributes that can be reliably evaluated in isolation [16, 18, 28, 36].

This intrinsic-versus-contextual distinction is a methodological necessity imposed by the nature of OSS issue titles, where project-specific details such as stakeholders, deadlines, and constraints are often missing. Assessing the remaining ISO attributes, specifically necessity, feasibility,

[11] https://openai.com/index/openai-o3-mini/

[12] https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-70B

[13] Metrics collected on March 22, 2025.

completeness, appropriateness, and correctness, requires domain goals and system-level information that are unavailable in this data source. Consequently, we exclude these context-dependent attributes, as they cannot be reliably evaluated without human expertise or rich project documentation [16, 18]. Restricting our analysis to intrinsic linguistic properties ensures that our evaluation remains robust and consistent with contemporary RE theory.

To evaluate the selected attributes, we applied a Likert scale ranging from 1 to 5, where 1 represents poor quality and 5 represents high quality. This five-point Likert-style scale was adopted to maintain consistency with prior LLM-based RE evaluation studies [2, 28], ensuring methodological comparability and facilitating replication. While absolute numerical comparisons across studies are limited by differences in task granularity, datasets, and evaluation criteria, relative trends, such as which prompting strategies enhance particular ISO 29148 attributes, remain meaningful and allow for conceptual comparison. This consistency supports both internal validity and interpretability of results in the broader context of emerging LLM-based RE evaluations.

It is important to acknowledge the limitations inherent in this textual assessment approach, particularly regarding verifiability. Consequently, some requirements may be classified as verifiable under our intrinsic, text-based criteria even though additional project-specific detail would be needed to fully verify them in a real engineering setting.

3.3.2 LLM-as-a-judge

To evaluate the large volume of software requirements generated by the different LLMs and prompt strategies employed in this study, we adopted the "LLM-as-a-Judge" paradigm. This methodology provides a scalable and cost-effective alternative to extensive human evaluation, which is often impractical for the large number of outputs generated in comparative LLM studies [55]. This choice is supported by recent empirical evidence suggesting that LLMs can serve as reliable proxies for human experts in technical domains. Zheng et al. [55] demonstrated that strong LLM judges can achieve agreement rates exceeding 80% with human preferences, a level comparable to the inter-rater reliability observed among human evaluators themselves. Furthermore, in the specific context of requirements engineering, Casciani et al. [7] found that LLM judges evaluating formal planning models achieved a Pearson correlation of $r = 0.97$ to $r = 0.99$ with expert human judges, validating their utility for assessing structural and semantic correctness.

A critical advantage of this approach for our study is consistency. As highlighted in a recent survey by Li et al. [20], human evaluation is susceptible to fatigue and inconsistency, particularly when processing large datasets. For our set of 900 requirements, maintaining human attention to consistently apply ISO/IEC/IEEE 29148 criteria would be methodologically challenging. The LLM judge eliminates this variance, ensuring that the definitions of Unambiguity, Verifiability, and Singularity are applied uniformly across all samples.

We utilized the `Qwen QwQ-32b` [41, 53] as the evaluator. This model was selected for its strong general-purpose reasoning performance, open availability, and architectural distinction from both generation models (`OpenAI o3-mini` and `DeepSeek R1`). This choice follows methodological recommendations in recent LLM-as-a-Judge studies [20, 55], which highlight the importance of using an evaluator that is not architecturally aligned with the generation models to mitigate self-enhancement bias.

It is important to clarify that the role of the evaluator is restricted to assessing the *intrinsic* linguistic quality attributes derived from ISO/IEC/IEEE 29148:2018 [25], as defined in the previous section. These attributes can be operationalized and judged without domain-specific knowledge or familiarity with project constraints, as they focus on textual clarity and testability rather than domain adequacy. Consequently, the evaluator model is not required to understand project terminology or stakeholder context, but rather to apply standardized criteria to individual requirement statements.

Employing an LLM as a judge in this context offers several advantages. It promotes consistent evaluation across all 900 generated requirements, as the LLM is not affected by fatigue or subjective variability that can influence human raters [20]. Another key benefit, as highlighted by Zheng et al. [55], is explainability: the judge can be configured to provide structured justifications for its assessments. Our implementation leveraged this feature by instructing the evaluator to return both scores and rationale aligned with the standard. This rationale is essential for interpreting quantitative patterns and addressing RQ3. Despite inherent LLM limitations, this approach provides a scalable, transparent, and reproducible means of evaluating requirement quality across large datasets.

While leveraging the LLM-as-a-Judge approach is advantageous for this study's scale, it is essential to acknowledge and mitigate potential limitations. LLM judges can exhibit inherent biases, such as favoring outputs from models with similar architectures or being influenced by the length or position of the text [20, 55]. To address the risk of self-enhancement bias, our deliberate selection of `Qwen QwQ-32b` ensures architectural distinctness from the models being tested. Furthermore, by providing the judge with clear, standard-based definitions in its prompt, we aimed

to constrain its decision-making process and improve construct validity. Despite these considerations, the LLM-as-a-Judge method was selected for its capacity to provide systematic scoring, enabling a robust comparative analysis of how LLM choice and prompt design influence the quality of the generated requirements artifacts.

3.3.3 Evaluation prompt

The prompt instructs the evaluator to apply the three quality criteria defined above, return a 1–5 score for each, and provide a textual rationale in a structured *JSON* schema. The complete prompt is as follows:

Prompt - LLM Evaluator

As an experienced requirements engineer, your task is to evaluate the quality of a given software requirement based on the following criteria:

Unambiguity: The requirement is stated in such a way so that it can be interpreted in only one way. The requirement is stated simply and is easy to understand. Score 1 if the requirement is highly ambiguous and 5 if there is no room for multiple interpretations.

Verifiability: The requirement is structured and worded such that its realization can be proven (verified) to the customer's satisfaction at the level the requirements exists. Verifiability is enhanced when the requirement is measurable. Score 1 if the requirement is difficult to verify, and 5 if it is easily measurable.

Prompt - LLM Evaluator

Singularity: The requirement states a single capability, characteristic, constraint or quality factor. Score 1 if the requirement includes several needs or is unclear in its focus, and 5 if it is entirely singular in nature.

For each criterion, provide a score from 1 to 5, with 1 being the lowest (poor) and 5 being the highest (excellent). Structure your output as a JSON object containing the scores and a combined explanation for your ratings.

While verifiability and measurability are distinct concepts, the latter serves as a practical indicator of the former in ISO/IEC/IEEE 29148:2018 [25]. The simplified phrasing "easily measurable" was intentionally adopted to ensure that the LLM evaluator could apply the criterion consistently, reflecting the standard's intent that verifiability depends on objective and demonstrable conditions rather than strictly quantitative metrics. This operationalization maintains conceptual alignment with the ISO definition while promoting scoring consistency across large scale automated evaluations.

The JSON Schema the model was instructed to follow is outlined below:

```
{
  "properties": {
    "unambiguity": {
      "description": "1-5 score for unambiguity",
      "title": "Unambiguity",
      "type": "integer"
    },
    "verifiability": {
      "description": "1-5 score for verifiability",
      "title": "Verifiability",
      "type": "integer"
    },
    "singularity": {
      "description": "1-5 score for singularity",
      "title": "Singularity",
      "type": "integer"
    },
    "explanation": {
      "description": "Combined textual explanation for the scores",
      "title": "Explanation",
      "type": "string"
    }
  },
  "required": [
    "unambiguity",
    "verifiability",
    "singularity",
    "explanation"
  ]
}
```

3.4 Data analysis strategy

To tackle the research questions, we will leverage the LLM-as-a-judge approach to examine the entire collection of 900 LLM-generated requirements, each annotated with Likert scale ratings for *Unambiguity*, *Verifiability*, and *Singularity* alongside the qualitative rationale of the judge, using a mixed-methods design. Quantitatively, we will compute descriptive statistics (means, medians, standard deviations, and frequency distributions) and visualize the score distributions, both in general and within the experimental groups defined by LLM, prompt style and repository. Because the data are ordinal and may violate normality assumptions, we will apply non-parametric tests: the Mann–Whitney *U* [37] test for pairwise comparisons (e.g., between LLMs) and the Kruskal–Wallis test [29], followed by Dunn's post hoc procedure [14] where appropriate (e.g., among prompt styles). Spearman's rank correlation coefficient [43] will be calculated to assess the associations between the three quality metrics.

Qualitatively, we performed a thematic analysis of the narrative feedback of judges recorded in the `Explanation` field, supplemented with illustrative excerpts from the `Requirement` field, to uncover recurring strengths, weaknesses, and patterns linked to specific combinations of LLM prompts. To manage the large volume of textual rationales, we first aggregated the 900 explanations into a structured dataset, tagging each entry with its corresponding LLM, prompt style, and repository. An initial open coding phase was conducted to identify recurring semantic categories (e.g., clarity, completeness, ambiguity, overspecification), followed by axial coding to group related categories into higher level themes describing common strengths and weaknesses. This process was performed iteratively and supported by spreadsheet based analysis, enabling traceability between each coded excerpt and its source requirement. While analyzing all rationales demanded considerable manual effort, the structured JSON format and short length of each explanation (typically 30–70 words) made the process tractable and ensured transparency in linking qualitative findings to quantitative results.

Integrating these qualitative insights with the quantitative findings provided a richer explanation of the factors influencing requirement quality. Together, these complementary analyses, statistical and thematic, enabled a comprehensive understanding of how LLMs and prompting strategies affect requirement quality.

Since our data did not meet normality assumptions, we applied non-parametric tests appropriate for ordinal scales. The Mann–Whitney U [37] and Kruskal–Wallis [29] tests are rank based methods widely used to compare independent groups without assuming normal distribution. When the Kruskal–Wallis test indicated significant differences, we used Dunn's post-hoc test for pairwise comparisons with Bonferroni correction. To examine associations among ordinal variables, we employed Spearman's rank correlation to measure monotonic relationships. For the qualitative component, we used Thematic Analysis [9], a structured approach for identifying and interpreting recurring patterns in open-ended responses. This involved an iterative coding process where model rationales were categorized to derive semantic themes regarding the quality of requirements. The combination of these methods follows analytical practices observed in recent studies [18, 23, 28, 36], ensuring methodological consistency with contemporary empirical research in Requirements Engineering.

Table 2 Overall statistics for requirement metrics (N = 900)

Statistic	Unambiguity	Verifiability	Singularity
Mean	4.34	4.25	4.38
SD	0.69	0.84	1.15
Min	2.00	1.00	1.00
Med	4.00	4.00	5.00
Max	5.00	5.00	5.00

4 Results and discussion

This section reports and discusses the results of evaluating Large Language Models for automatically generating software requirements from GitHub issue titles. Using two LLMs and three prompt engineering strategies, we produced 900 candidate requirements from 150 feature requests drawn from five open source repositories. Each requirement was rated by an LLM-as-a-Judge on three ISO/IEC/IEEE 29148:2018 quality attributes, *Unambiguity*, *Verifiability*, and *Singularity*, using a five-point Likert scale. The following subsections detail the overall quality observed, address each research question through quantitative and qualitative analysis integrated with discussion points, and present additional relevant findings.

4.1 Overview of requirement quality

An initial analysis across all 900 generated requirements established a baseline understanding of the overall quality achieved by the LLMs. The key descriptive statistics for *Unambiguity*, *Verifiability*, and *Singularity* are summarized in Table 2.

Overall, the LLM judge rated the generated requirements favorably, with mean scores exceeding 4.25 and median scores of 4 or 5 across all metrics. This suggests that LLMs can often produce requirements perceived as largely unambiguous, verifiable, and singular according to the ISO 29148-based definitions employed in this study, aligning

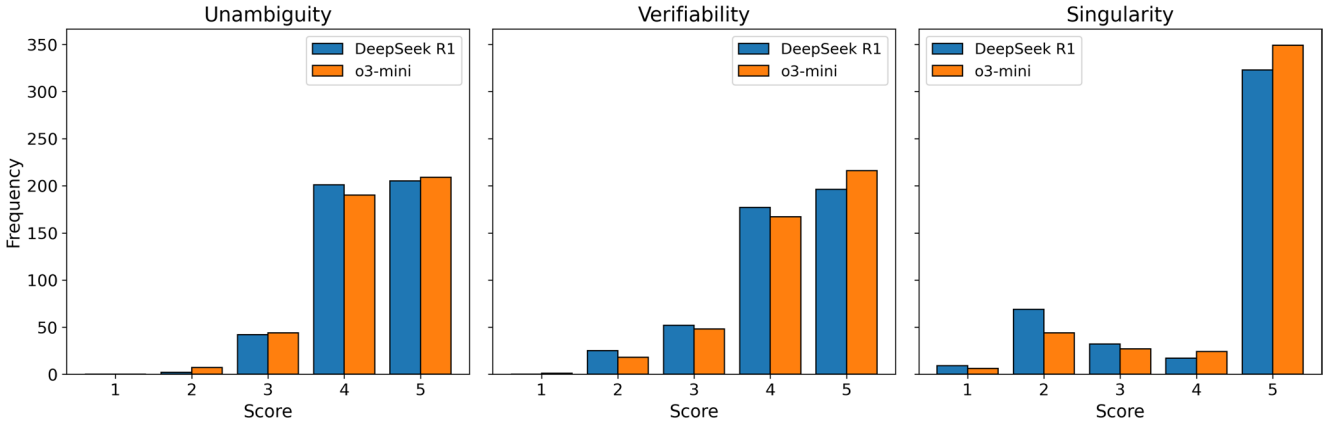


Fig. 2 Distribution of Requirement Quality Scores by Metric and LLM

Table 3 Descriptive statistics for quality metrics by LLM (N=450 per LLM)

	o3-mini			DeepSeek		
Metric	Mean	Med	SD	Mean	Med	SD
Unambiguity	4.34	4	0.72	**4.35**	4	0.67
Verifiability	**4.29**	4	0.83	4.21	4	0.86
Singularity	**4.48**	5	1.06	4.28	5	1.23

with the potential highlighted by [4, 38]. However, the standard deviations, particularly for *Singularity* ($SD = 1.15$), and minimum scores indicate significant variability and occasional deficiencies, underlining the challenges noted in the background regarding the inherent difficulties in achieving consistent quality [17].

The distribution of scores (Fig. 2) confirms this: while high scores (4 and 5) dominate, a non-trivial tail of lower scores exists, especially for *Verifiability* and *Singularity*. This variability suggests that while LLMs show promise, achieving consistent, high-quality requirement generation, particularly for verifiability and singularity which are crucial for downstream activities, remains a challenge influenced by factors explored below.

4.2 RQ1: LLM effectiveness in requirements generation

Comparing the effectiveness of the two LLMs (Table 3 and Fig. 2), both `o3-mini` and `DeepSeek` demonstrated comparable capabilities in generating generally high-quality requirements from issue titles, with median scores of 4 for *Unambiguity* and *Verifiability*, and 5 for *Singularity*.

While average scores were similar, Mann–Whitney U tests revealed a statistically significant advantage for `o3-mini` in generating requirements with better *Singularity* ($U = 108009.5$, $p = 0.023$), suggesting it was slightly more adept at focusing on a single purpose per statement. No significant differences were found for *Unambiguity* ($p = 0.967$) or *Verifiability* ($p = 0.156$). The substantial standard deviations observed for both models, however, underscore that the choice of LLM is only one factor; input quality and prompting strategy likely exert considerable influence, aligning with the general understanding that LLM performance is context-dependent.

While our mean scores suggest a generally high level of effectiveness, directly comparing these numerical results to other studies in LLM-based RE is challenging due to differences in task granularity and evaluation criteria.

Despite these differences, our findings conceptually converge with the broader literature on two key points. First, LLMs demonstrate significant promise as capable assistants for drafting RE artifacts. Our high mean scores align with the favorable results reported by both Krishna et al. [28], where CodeLlama produced outputs comparable to a human benchmark, and Almonte et al. [2], where generated NFRs achieved a median validity score of 5.0/5.0. Second, and crucially, all studies report significant variability and notable weaknesses in the generated outputs. The substantial standard deviations in our results (e.g., 1.15 for *Singularity*) and the presence of low-scoring requirements mirror the findings of Krishna et al. [28], who noted issues with verbosity and flawed formatting, and Almonte et al. [2], who found that nearly 20% of generated NFRs had mismatched quality attributes. This shared conclusion underscores that while LLMs are effective, their output is not uniformly perfect, reinforcing the need for careful evaluation and refinement.

Table 4 Descriptive statistics for `o3-mini` scores by prompt style (N=150 per style)

	Naive			Few-shot			Expert		
Metric	Mean	Med	SD	Mean	Med	SD	Mean	Med	SD
Unambiguity	4.37	4	0.70	**4.43**	5	0.76	4.21	4	0.67
Verifiability	4.21	4	0.82	4.29	5	0.97	**4.36**	4	0.67
Singularity	4.56	5	0.97	**4.77**	5	0.75	4.11	5	1.29

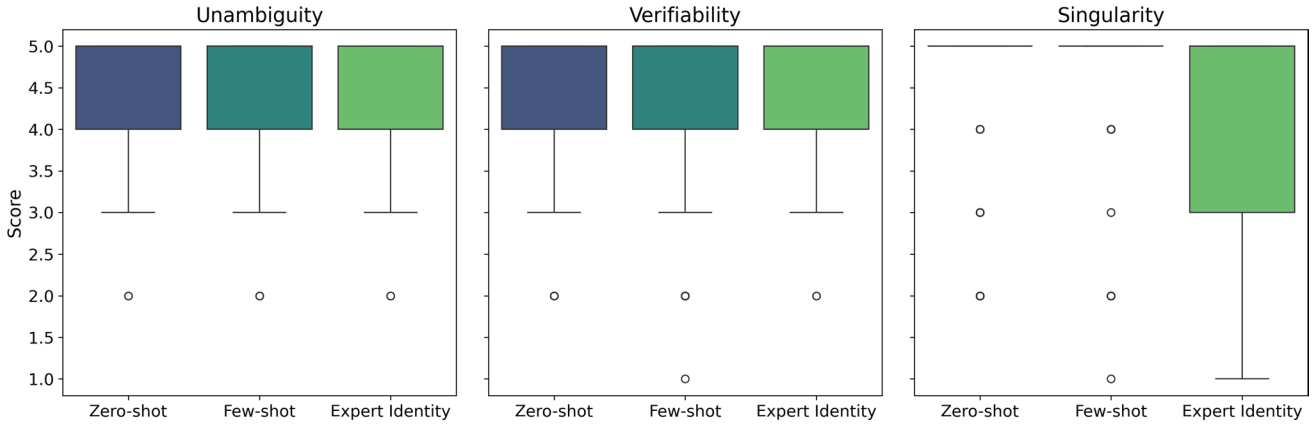


Fig. 3 Distribution of `o3-mini` Requirement Quality Scores by Prompt Style

Table 5 Descriptive statistics for `DeepSeek R1` scores by prompt style (N=150 per style)

	Naive			Few-shot			Expert		
Metric	Mean	Med	SD	Mean	Med	SD	Mean	Med	SD
Unambiguity	4.31	4	0.65	4.33	4	0.74	**4.42**	4	0.59
Verifiability	4.18	4	0.79	4.07	4	0.98	**4.37**	5	0.76
Singularity	4.20	5	1.27	**4.69**	5	0.86	3.95	5	1.37

RQ1 Summary

LLMs are generally effective at generating software requirements from OSS issue titles, achieving high mean quality scores ($> 4.2/5$) for Unambiguity, Verifiability, and Singularity. However, this effectiveness is inconsistent, with significant score variability indicating that high quality is not guaranteed and is dependent on other factors like input clarity. The two tested models performed comparably overall, though `o3-mini` held a minor but statistically significant advantage in producing requirements with better Singularity.

4.3 RQ2: Influence of prompt engineering strategies

Investigating the influence of prompt engineering (RQ2) confirmed its significant impact, though the effects were notably model-dependent, reinforcing findings from [42, 50] about the importance of prompt design while adding nuance about model-specific responses.

4.3.1 Prompt influence on o3-mini

For `o3-mini` (Table 4, Fig. 3), Kruskal–Wallis tests showed significant prompt effects on *Unambiguity* ($H = 12.50$, $p = 0.002$) and *Singularity* ($H = 30.08$, $p < 0.001$). The `Few-shot` prompt yielded the best results for both, achieving the highest median (5) for *Unambiguity* and the highest mean (4.77) for *Singularity*. Conversely, the `Expert` prompt, despite its intention to leverage simulated domain knowledge [52], underperformed, particularly degrading *Singularity* (mean 4.11, SD 1.29). This suggests that for `o3-mini`, providing concrete examples (Few-shot) was more effective than assigning a persona.

Although the Kruskal–Wallis test detected a statistically significant effect of prompt style on Unambiguity for `o3-mini`, the boxplot in Fig. 3 shows that the distributions are visually similar. Table 4 clarifies that this significance is driven by a modest upward shift in the median for the Few-shot prompt (5.0) compared to Naive and Expert (4.0), rather than a large separation in means. In contrast, the trade-off for Singularity is clearly visible: the Expert prompt exhibits the highest dispersion (SD = 1.29) and lowest mean (4.11), visually confirming that detailed persona instructions can compromise the atomicity of the requirement.

4.3.2 Prompt influence on DeepSeek R1

For `DeepSeek R1` (Table 5, Fig. 4), significant prompt effects were found for *Verifiability* ($H = 7.84$, $p = 0.020$)

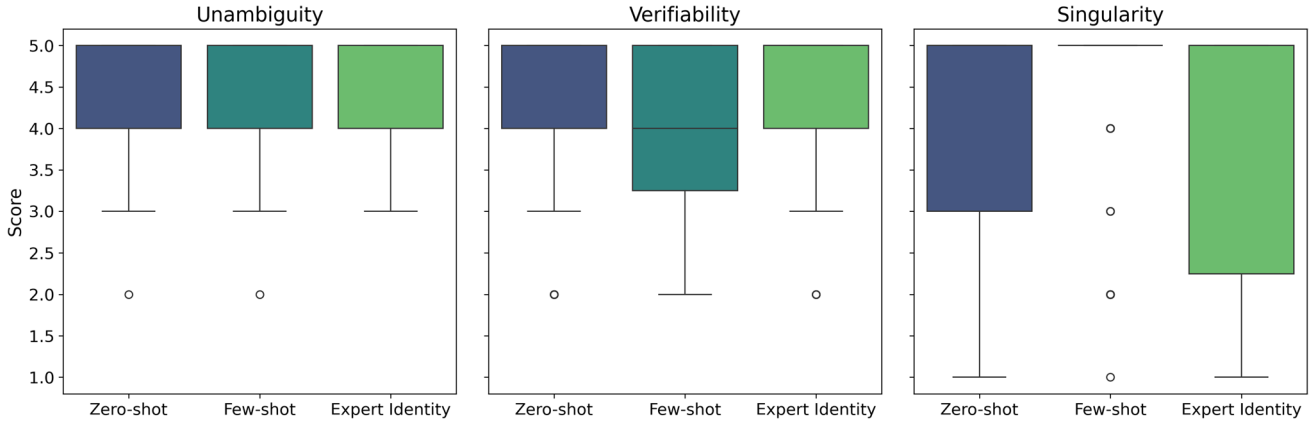


Fig. 4 Distribution of `DeepSeek R1` requirement quality scores by prompt style

and *Singularity* ($H = 29.00, p < 0.001$). Here, the `Expert` prompt significantly improved *Verifiability* (median 5, mean 4.37). However, mirroring the `o3-mini` results, the `Expert` prompt led to the lowest mean *Singularity* score (3.95) and highest variability (SD 1.37). The `Few-shot` prompt was again most effective for enhancing *Singularity* (mean 4.69, SD 0.86). This highlights a potential trade-off, particularly with the Expert prompt: attempts to increase detail for verifiability might encourage the model to combine multiple facets, reducing singularity.

For `DeepSeek R1`, Fig. 4 shows that while Verifiability exhibits a statistically significant improvement under the Expert prompt ($H = 7.84, p = 0.020$), the visual difference is subtle. The effect is driven more by a relative decline in Few-shot performance than by a strong absolute gain in Expert. In contrast, Singularity reveals a more pronounced variation: the Naive prompt yields the lowest medians and largest spread, indicating inconsistent atomicity, whereas the Few-shot configuration produces tighter clustering and higher central tendency. These patterns visually substantiate the small yet systematic effects identified by the non-parametric tests, confirming that prompt influence differs not only in magnitude but also in direction across models.

RQ2 Summary

Prompt engineering significantly influences the quality of generated requirements, but its effects are model-dependent and can introduce critical trade-offs between quality attributes. The `Few-shot` strategy consistently and significantly improved Singularity for both LLMs, proving effective for generating focused, atomic requirements. Conversely, the `Expert Identity` prompt created a trade-off: it sometimes enhanced Verifiability by adding detail (notably for `DeepSeek R1`), but frequently did so at the cost of degrading Singularity for both models.

4.4 RQ3: Qualitative characteristics, strengths, and weaknesses

Qualitative review of the generated requirements and judge explanations provided deeper insights (RQ3), largely corroborating the quantitative findings.

4.4.1 General patterns

LLMs consistently produced syntactically well-formed requirement statements adhering to basic requirement structure (e.g., "The system shall..."). Effectiveness strongly depended on the clarity of the input issue title, consistent with the well-known "garbage-in, garbage-out" principle and the known challenges of dealing with ambiguous natural language inputs in RE [5, 17]. Capturing core intent was generally successful for clear titles, but achieving adequate Verifiability was a common weakness, especially when titles lacked specific detail.

4.4.2 Prompt/LLM specifics

Consistent with quantitative results, `o3-mini` generally produced more focused, singular requirements. The `Naive` prompt often yielded direct but potentially underdeveloped rephrasing. The `Few-shot` prompt encouraged adherence to a standard, singular format, sometimes at the cost of nuance. The `Expert` prompt often led to more elaborate outputs; for `DeepSeek R1`, this sometimes improved Verifiability by adding (occasionally speculative) detail, but frequently harmed Singularity for both models by combining multiple aspects into one statement (see illustrative Example 2 and 4 below).

However, deeper examination of the examples revealed that the `Expert` prompt's higher Verifiability scores occasionally stemmed from speculative elaborations that were not grounded in the original issue titles. In Example

2, for instance, the model expanded the requirement with implementation oriented assumptions that did not explicitly appear in the source request. While such elaborations increase measurability and surface completeness, they risk introducing details that may be inconsistent with the actual system context or overly constrain the solution space. This behavior suggests that the `Expert` configuration can act as a creativity enhancing mechanism rather than a precision driven specification tool. Therefore, its use in real world RE tasks should be paired with human validation to ensure factual correctness and contextual alignment, preventing the inclusion of "hallucinated" or misaligned requirements.

To illustrate these observations, consider the following examples extracted from the dataset. In these illustrative examples, we adopt the notion of *intrinsic linguistic verifiability*, that is, whether a requirement states a behavior that can in principle be checked or tested based on its wording, rather than full project level verifiability, which would require contextual information that is not available from OSS issue titles.

4.4.3 Example 1 (perfect score - `o3-mini`, `Few-shot`):

Issue Title: `Support for 'uint16', 'uint32', and 'uint64'` (pytorch/pytorch, 58734)
Requirement: The system shall support data types uint16, uint32, and uint64.
Scores (U, V, S): 5, 5, 5
Judge Explanation: The requirement clearly specifies exact data types (uint16, uint32, uint64) without ambiguity. It is a standalone statement focusing on a single capability (data type support) and can be verified through implementation checks.
Observation: This result demonstrates the superiority of `Few-shot` prompting for handling precise technical inputs. Notably, when the same model processed this identical request using the `Expert` prompt, the scores dropped to 4 for both Unambiguity and Verifiability. This indicates that the Expert persona introduced unnecessary elaboration that diluted the clarity of the original request, whereas the `Few-shot` strategy ensured a high-fidelity transformation[14]

4.4.4 Example 2 (low singularity – `DeepSeek R1`, `Expert`):

Issue Title: `Implement Terrain` (godotengine/godot-proposals, 6121)
Requirement: The system shall provide a built-in terrain system that allows users to create, modify, and manage 3D landscapes within the Godot engine. The terrain system must support heightmap-based terrain generation, texture painting with multiple layers, and integration with Godot's physics, lighting, and material systems. It should include features for painting vegetation, placing water, and handling level-of-detail (LOD) for performance optimisation. The system must also provide a terrain editor tool with a user-friendly interface for height, texture, and material adjustments, and ensure compatibility with existing Godot features such as dynamic lighting and shadows. Additionally, the terrain system should be designed to be efficient, scalable, and extensible to accommodate large open-world environments and user-defined customizations.
Scores (U, V, S): 3, 3, 2
Judge Explanation: The requirement specifies a terrain system with clear features (LOD management, dynamic streaming, physics integration, editor tools, extensibility). However, terms like 'high-performance', 'large-scale', and 'intuitive' introduce ambiguity. Verifiability is achievable for most features (e.g., presence of tools, shared integration), but subjective terms like 'seamless workflow' complicate testing. The requirement focuses solely on the terrain system, covering its technical and tooling aspects without mixing unrelated concerns.[15]
Observation: This example provides a clear qualitative illustration of the trade-off identified in our quantitative results. As shown in Tables 4 and 5, the Expert prompt yielded the lowest mean Singularity scores for both the `o3-mini` and `DeepSeek R1` models, while simultaneously improving Verifiability for `DeepSeek R1`. This case demonstrates precisely how that trade-off can manifest. The underlying cause likely stems from a conceptual tension between the prompt's instructions and the definition of the Singularity attribute. The prompt asks the LLM to act as an 'expert' and generate a 'complete, and actionable' requirement. In response to such a complex input title, the model attempts to be comprehensive, bundling multiple related sub-features into a single, detailed statement. While this detail can enhance Verifiability by specifying concrete functionalities, it directly violates the principle of Singularity. The fact that this trend was observed quantitatively across both LLMs and a diverse set of 150 inputs strengthens the conclusion that this is a systemic effect of the Expert prompt's design, rather than an artifact of a specific model or input.

[14] All requirement variations generated in this study are available in the full dataset on Zenodo: https://doi.org/10.5281/zenodo.15003691.

[15] The low Singularity score appears inconsistent with the judge narrative, suggesting either judgment error or a more nuanced interpretation.

4.4.5 Example 3 (vagueness from vague title – `o3-mini`, `Naive`):

> *Issue Title:* `Reusing state logic is either too verbose or too difficult` (flutter/flutter, 51752)
> *Requirement:* The system MUST include a mechanism that simplifies the reuse of state logic, minimizing verbosity and reducing complexity in implementation.
> *Scores (U, V, S):* 3, 2, 5
> *Judge Explanation:* The requirement specifies a mechanism to simplify state logic reuse with clarity on purpose but uses vague terms like 'minimizing' and 'reducing' without quantifiable metrics. It focuses on a single objective.
> *Observation:* This outcome demonstrates that the 'garbage-in, garbage-out' principle is amplified by the Naive prompt. The failure here is not just in the input, but in the zero-shot strategy's inability to guide the model from problem space to solution space, a crucial step in requirements specification.

4.4.6 Example 4 (improved verifiability / low singularity – `DeepSeek R1`, `Expert`):

> *Issue Title:* `Use mold as default linker?` (rust-lang/rust, 94347)
> *Requirement:* The Rust compiler (`rustc`) shall integrate mold as the default linker on systems where mold is supported. The build system must detect the availability of mold and automatically select it over other linkers during the linking phase, while still providing a fallback option for environments that do not support mold. This integration must include comprehensive testing to ensure performance gains and maintain correctness, as well as documentation and configuration options that allow users to override the default behavior if necessary.
> *Scores (U, V, S):* 4, 5, 2
> *Judge Explanation:* The requirement specifies clear integration steps, detection mechanism, fallback, testing, documentation, and configuration. While it contains multiple components, each part is clearly defined. Unambiguity is high as terms are technically precise. Verifiability is strong due to measurable aspects like detection, selection, and testing. Singularity is low because the requirement combines integration, detection, fallback, testing, documentation, and configuration into a single statement.
> *Observation:* This behavior contrasts sharply with what a Naive or Few-shot prompt would likely produce, a singular but far less useful statement. Such a requirement, while scoring high on Singularity, would score poorly on Verifiability because it ignores crucial, practical questions that must be resolved before implementation. The Expert prompt, therefore, pushes the LLM to generate what is functionally closer to a small-scale feature specification rather than an atomic requirement. This reveals a critical tension for practitioners using LLMs in RE: prompts designed to elicit detailed, actionable, and verifiable outputs may inherently conflict with the fundamental principle of requirement atomicity, producing outputs that require manual decomposition into multiple, truly singular requirements.

It is important to note that several of the verification related aspects described in Example 4 such as fallback mechanisms, configuration options, and testing procedures could alternatively be expressed as acceptance criteria rather than within the requirement statement itself. While their inclusion improves perceived Verifiability, it also blurs the boundary between requirement specification and acceptance definition, a distinction emphasized in RE guidelines such as ISO/IEC/IEEE 29148:2018. Furthermore, our experimental design assumed a one-to-one correspondence between each issue title and a single requirement to ensure methodological consistency across LLMs and prompts. However, in practical OSS environments, individual issues often encapsulate multiple requirements or sub-features. Future research could address this by adopting a multi-level mapping approach distinguishing between atomic requirements, grouped functionalities, and corresponding acceptance criteria to more accurately reflect real world RE practice.

RQ3 Summary

Qualitatively, a key strength of LLMs is their ability to capture the core intent from clear input titles and produce syntactically correct requirements. Their primary weakness is a strong dependency on input clarity, where vague or overly broad titles invariably yield requirements that lack Verifiability. A recurring characteristic is a fundamental trade-off between detail and focus; prompts designed to elicit comprehensive content (e.g., `Expert`) often produce more descriptive statements that violate the principle of Singularity by bundling multiple distinct capabilities.

Although our analysis primarily emphasized the "garbage-in, garbage-out" phenomenon, an exploratory review of the dataset revealed indicative patterns in how issue title characteristics influenced LLM performance. Titles containing explicit actions and concrete entities (e.g., "Add support for...", "Implement feature X...", "Enable configuration parameter...") were more likely to produce requirements rated high in *Unambiguity* and *Verifiability*. Conversely, vague or evaluative titles (e.g., "Improve usability", "Make it faster") often yielded underspecified or subjective outputs that reduced *Verifiability* and sometimes led to ambiguous

phrasing. These observations suggest the following practical recommendations for OSS contributors writing issue titles intended to serve as inputs for automated requirement generation: (i) use an explicit action verb followed by a concrete entity (e.g., "Implement...", "Add support for..."); (ii) avoid subjective or evaluative language that lacks measurable criteria; and (iii) include the specific component, parameter, or scope affected. While a systematic correlation analysis was beyond this study's scope, these preliminary trends suggest that linguistic specificity in issue formulation may play a key role in shaping the strengths and weaknesses of LLM-generated requirements. Future research could extend this analysis by quantitatively examining lexical or syntactic features of issue titles as predictors of LLM performance and prompt sensitivity.

4.5 Additional findings

Supplementary analyses revealed statistically significant variations in requirement quality based on the source repository for all three metrics ($p < 0.001$ via Kruskal–Wallis), reinforcing the critical role of input data quality [33] and differing community practices in writing feature requests. Spearman correlation analysis (Table 6) showed a strong positive relationship between *Unambiguity* and *Verifiability* ($\rho = 0.69$), consistent with ISO standards [25] where clarity enables verification. Correlations involving *Singularity* were weaker ($\rho = 0.37$ with Unambiguity; $\rho = 0.30$ with Verifiability), highlighting that while related, these attributes capture distinct quality dimensions.

An important implication of our findings relates to the validity of the LLM-as-a-Judge methodology itself. Our results consistently demonstrated that the Expert prompt encouraged the generation of more verbose and elaborate requirements, which in turn received lower scores for Singularity (see Sect. 4.3). This observation is particularly noteworthy when viewed in the context of known limitations of automated evaluation. Prior research, such as that by Zheng et al. [55], has identified a significant "verbosity bias", where LLM judges tend to unfairly favor longer responses. Interestingly, our study suggests that the evaluation protocol successfully resisted this bias for the Singularity metric. Rather than rewarding length, our judge correctly penalized the less-focused, multi-faceted requirements that resulted from the Expert prompt's verbosity. This robustness is likely attributable to the explicit, standard-based definition for Singularity provided in the evaluation prompt, which instructed the judge to score requirements based on whether they stated a single capability.

This suggests that grounding LLM-as-a-Judge evaluations in explicitly operationalized criteria derived from established standards like ISO/IEC/IEEE 29148:2018 rather than assuming those criteria are inherently clear can improve interpretability. Furthermore, the reliability of the judge's rationale is supported by recent findings from Zheng et al. [55], who reported that when humans and LLM judges initially disagreed, humans deemed the LLM's explanation reasonable in 75% of cases upon review. This reinforces the credibility of the qualitative insights derived in RQ3, indicating that the evaluator's feedback is not arbitrary but grounded in a consistent logic that withstands scrutiny.

Table 6 Spearman correlation matrix for quality metrics ($N = 900$)

Metric	Unambiguity	Verifiability	Singularity
Unambiguity	1.00	0.69	0.37
Verifiability	0.69	1.00	0.30
Singularity	0.37	0.30	1.00

4.6 Human validation study

To validate the reliability of the LLM-as-a-Judge methodology adopted in this study and to strengthen the internal validity of our findings, we conducted a targeted human validation study following established practices in empirical Requirements Engineering research [2, 7, 28, 36]. The goal of this study is to assess whether the automated scores assigned by the Qwen QwQ-32b judge constitute a reliable proxy for human expert judgment on the three intrinsic ISO/IEC/IEEE 29148:2018 quality attributes: *Unambiguity*, *Verifiability*, and *Singularity*.

4.6.1 Sample selection

From the full set of 900 generated requirements, we selected a stratified random sample of 50 requirements (5.6% of the dataset). This sample size is consistent with prior RE validation studies: Almonte et al. [2] validated 34 representative samples from 1,593 generated non-functional requirements (≈2.1%), while Lubos et al. [36] and Krishna et al. [28] each employed four human evaluators on complete but small-scale requirement sets. While relatively small, this sample is generally considered sufficient for agreement analysis in RE studies involving ordinal ratings, where the focus lies on inter-rater consistency rather than population-level estimation.

The stratified sampling was designed along two dimensions to ensure representativeness of the full experimental design:

(i) *Experimental cell:* requirements were drawn from all six combinations of LLM (2) × prompt style (3), with an approximately balanced distribution across cells (8–9 requirements per cell);
(ii) *LLM-judge score distribution:* within each cell, requirements were sampled to include low (average score ≤

3.0), medium (3.0 < average $\leq$ 4.0), and high (average > 4.0) scoring items, with a minimum of one low-scoring requirement per cell to avoid ceiling-effect bias.

This deliberate inclusion of requirements across the full score spectrum strengthens the robustness of the validation, as it enables the detection of systematic biases in the LLM judge, particularly potential leniency bias identified in the literature [20].

4.6.2 Evaluators

Five human evaluators independently assessed all 50 requirements. All evaluators are professionals currently working in the software industry, while also pursuing graduate studies (three PhD candidates and two Master's students) in Software Engineering or Requirements Engineering. Their professional experience in software development ranges from 3 to over 10 years, with practical involvement in Requirements Engineering activities such as elicitation, specification, analysis, and validation. All evaluators had prior familiarity with ISO/IEC/IEEE 29148 and experience writing or reviewing Software Requirements Specification documents in real-world projects. This combined academic and industry profile strengthens the validity of the evaluation, as it reflects practitioners with both theoretical grounding and practical expertise. Furthermore, this profile is consistent with comparable RE validation studies: Krishna et al. [28] employed four reviewers from academia and industry with at least three years of experience, Lubos et al. [36] recruited participants with "educational and professional backgrounds in software engineering" without requiring RE expertise, and Almonte et al. [2] recruited ten industry software quality professionals. Table 7 summarizes the evaluators' profiles.

4.6.3 Evaluation protocol

The evaluation followed a two-phase protocol designed to improve consistency and reduce potential bias:

Phase 1—Calibration. Before the scored evaluation, each evaluator received a guideline document containing the *same* operational definitions of Unambiguity, Verifiability, and Singularity used in the LLM evaluation prompt (see Section 3.3), along with the 1–5 Likert scale anchors. Each evaluator then independently rated five calibration examples drawn from outside the 50-sample set. The authors reviewed the calibration ratings and identified interpretation divergences across evaluators; these were resolved through an asynchronous written exchange between the authors and the evaluators, after which the guideline document was updated to incorporate the agreed-upon interpretations. This iterative calibration procedure follows the recommendation of He et al. [21] to establish clear annotation guidelines prior to evaluation.

Phase 2—Independent evaluation. Each evaluator independently rated all 50 requirements on the three quality criteria using the same 1–5 Likert scale. Requirements were presented in a *randomized order that was unique to each evaluator* to mitigate position bias, a known limitation of both human and LLM-based evaluation [20, 30]. Evaluators were *blinded* to: (a) which LLM generated each requirement, (b) which prompt strategy was used, and (c) the scores assigned by the LLM judge. An optional free-text justification field was included for each rating. This design directly follows the "independent assessment" protocol of Lubos et al. [36], who demonstrated that blind evaluation provides an unbiased baseline for measuring human–LLM agreement. The evaluation guideline (including calibration examples and post-calibration alignment directives), individual evaluator ratings (anonymized as E1–E5), and aggregate agreement metrics are all available in the replication package to support transparency and reproducibility.

Table 7 Profile of human evaluators

ID	Role	Industry experience	RE experience	Education
E1	Software engineer	10+ years	5+ years	PhD Student
E2	Software engineer	8 years	4 years	PhD Student
E3	Requirements engineer	6 years	3 years	PhD Student
E4	Requirements engineer	4 years	2 years	Master's Student
E5	Software developer	3 years	<2 years	Master's Student

4.6.4 Agreement metrics

To comprehensively assess the reliability and potential bias of the LLM-as-a-Judge methodology, we computed the following agreement metrics, selected based on their widespread use in prior LLM evaluation and Requirements Engineering studies. These metrics capture complementary aspects of agreement, including inter-rater reliability, rank consistency, pairwise agreement, and systematic bias:

(i) *Krippendorff's alpha (ordinal)* [27]: inter-rater reliability among the five human evaluators, computed per quality attribute. Values ≥ 0.667 indicate acceptable agreement and values ≥ 0.800 indicate strong agreement [27];

(ii) *Spearman's rank correlation* (ρ) [43]: monotonic association between the mean human scores and the LLM-judge scores for each criterion, enabling direct comparison with the Pearson correlations ($r = 0.97$

Table 8 Summary of human–LLM agreement metrics per quality attribute

Criterion	Kripp. α	Spearman ρ	p-value	Mean κ_w	Exact	±1	$\overline{\Delta}$
Unambiguity	0.287	0.326	0.021	0.286	42%	84%	+0.36
Verifiability	0.150	0.424	0.002	0.293	34%	96%	+0.16
Singularity	0.394	0.488	<0.001	0.372	42%	72%	+0.20

κ_w: quadratic-weighted Cohen's kappa (mean over 5 evaluators). $\overline{\Delta}$: mean signed deviation (LLM − human)

–0.99) reported by Casciani et al. [7] in an RE-adjacent domain;

(iii) *Cohen's weighted kappa* [8]: pairwise agreement between each individual evaluator and the LLM-as-a-Judge, accounting for ordinal proximity in disagreements. This metric was adopted following Lubos et al. [36], who reported Cohen's kappa values ranging from $\kappa = 0.05$ (independent) to $\kappa = 0.75$ (bound) in their RE quality assessment study;

(iv) *Percentage agreement (exact and* ±1**)**: an interpretable baseline metric, benchmarked against the >80% agreement threshold established by Zheng et al. [55] between GPT-4 and human judges, and the 80.4% reported by Almonte et al. [2];

(v) *Signed mean deviation*: systematic bias direction and magnitude, computed as $\overline{\Delta} = \text{Score}_{\text{LLM}} - \bar{R}_{\text{Human}}$ per criterion, following the deviation analysis of Krishna et al. [28] to determine whether the LLM judge systematically inflates or deflates scores.

4.6.5 Human validation results

Table 8 summarizes the agreement metrics across the three quality attributes.

Inter-rater reliability. Krippendorff's alpha values were low for all three attributes ($\alpha = 0.150$–0.394), falling below the $\alpha \geq 0.667$ threshold for acceptable agreement [27]. This indicates substantial disagreement among the five human evaluators themselves, with Verifiability showing the lowest consensus ($\alpha = 0.150$) and Singularity the highest ($\alpha = 0.394$).

Human–LLM correlation. Despite the low inter-rater reliability, the Spearman rank correlations between mean human scores and LLM-as-a-Judge scores were statistically significant for all three criteria: Unambiguity ($\rho = 0.326$, $p = 0.021$), Verifiability ($\rho = 0.424$, $p = 0.002$), and Singularity ($\rho = 0.488$, $p < 0.001$). These moderate positive correlations indicate that the LLM judge's relative ranking of requirements is moderately aligned with the aggregate human assessment. This suggests that, despite individual disagreement, the overall ranking of requirement quality remains consistent between human evaluators and the LLM-as-a-Judge.

Pairwise agreement. The mean quadratic-weighted Cohen's kappa between individual evaluators and the LLM-as-a-Judge ranged from $\kappa_w = 0.286$ (Unambiguity) to $\kappa_w = 0.372$ (Singularity), indicating fair agreement according to Landis and Koch's scale. Individual evaluator–LLM kappa values ranged from $\kappa_w = 0.094$ to $\kappa_w = 0.488$, reflecting the variability also observed among human raters.

Percentage agreement. When comparing the human median score to the LLM-judge score, within-one-point agreement (±1) reached 84% for Unambiguity, 96% for Verifiability, and 72% for Singularity. Exact agreement was lower (34–42%), which is expected given the subjectivity of ordinal scales and the inherent ambiguity in evaluating textual quality attributes.

Systematic bias. The signed mean deviation was positive for all three attributes ($\overline{\Delta} = +0.16$ to $+0.36$), indicating a slight tendency of the LLM-as-a-Judge to assign higher scores than the human median. The largest bias was observed for Unambiguity ($\overline{\Delta} = +0.36$), while Verifiability showed the smallest ($\overline{\Delta} = +0.16$).

4.6.6 Human validation discussion

The human validation study reveals two complementary findings. First, the low Krippendorff's alpha values demonstrate that *requirement quality assessment is inherently subjective*, even among evaluators with relevant training and experience. This result is consistent with Lubos et al. [36], who reported Cohen's kappa values as low as $\kappa = 0.05$ for independent human evaluators assessing RE quality attributes, and reinforces the well-documented challenge of achieving high inter-rater agreement on subjective textual properties [27], which limits the maximum achievable agreement in this type of evaluation task.

Second, despite this human disagreement, the LLM-as-a-Judge's scores show statistically significant positive correlations with the aggregate human judgment across all three attributes ($p < 0.05$). Although the strength of the correlations is moderate, the consistency in relative ranking supports the practical usefulness of the LLM-as-a-Judge. The within-±1 agreement rates (72–96%) indicate that the LLM-as-a-Judge generally does not deviate from the human consensus by more than one point on the 5-point scale. This level of practical agreement is comparable to the >80% human LLM agreement threshold reported by Zheng et al. [55] and the 80.4% agreement found by Almonte et al. [2] in their RE quality study.

The mild positive bias ($\overline{\Delta} = +0.16$ to $+0.36$) indicates a slight leniency tendency in the LLM judge, consistent with the positional and leniency biases documented in the LLM-as-a-Judge literature [20]. However, this bias is modest (less than half a point on a 5-point scale) and systematic, meaning it can be anticipated and accounted for when interpreting scores.

These results provide empirical evidence that the LLM-as-a-Judge approach adopted in this study can serve as a practically useful and reasonably aligned approximation of human expert judgment on intrinsic requirement quality. The LLM judge performs comparably to an individual human evaluator whose agreement with peers is itself limited while offering perfect consistency and scalability across the full set of 900 requirements.

4.7 Overall discussion

This study demonstrates the potential of LLMs, evaluated using the scalable LLM-as-a-Judge approach [20, 55], to automate the generation of initial software requirements directly from unstructured feature requests in OSS repositories, addressing a known challenge in data-driven RE [33] and potentially alleviating difficulties associated with ambiguous natural language specifications [17]. Our findings underscore that while LLMs can generally produce high-quality outputs, their effectiveness is significantly modulated by prompt engineering, validating the focus of recent work [42, 50]. Crucially, the impact of prompting is model-specific, and strategies such as assigning an "Expert Identity" can introduce trade-offs, potentially improving Verifiability at the cost of Singularity. The strong dependence on the clarity of the input title and the observed variability highlight that LLMs are not a silver bullet for RE challenges [4, 38]. Rather, they serve best as powerful assistants requiring careful prompt design and human oversight to refine outputs, particularly ensuring adequate Verifiability and appropriate Singularity before use in development workflows. Future work should explore more sophisticated prompting strategies, methods for handling vague inputs, and integration of domain knowledge beyond simple persona prompts.

In our setting, the requirements generated by LLMs should be interpreted as draft artifacts with intrinsic textual quality, rather than as complete or context-appropriate specifications. Their practical usefulness still depends on human-led refinement, stakeholder negotiation, and domain specific interpretation, especially because issue titles do not provide the information needed to assess contextual ISO attributes such as completeness, necessity, or correctness. For the same reason, our evaluation methodology is not intended to replace domain level assessment of requirement correctness or feasibility. The LLM judge only needs to apply standardized ISO-based criteria to individual statements under minimal context, whereas domain validation remains a fundamentally human centered activity.

From a practical perspective, we therefore envision these models as assistive tools that help requirements engineers bootstrap structured candidate statements from noisy OSS artifacts. Any adoption in real-world development workflows should treat the generated requirements as starting points that must be reviewed, adapted, and prioritized in light of project goals, constraints, domain restrictions, and stakeholder preferences.

Recent prompting and reasoning strategies, such as Chain-of-Thought (CoT) prompting [48], complex instruction templates [50], and Retrieval-Augmented Generation (RAG) [15], have shown promise in improving reasoning transparency and factual grounding by incorporating intermediate reasoning steps or external knowledge sources. Integrating such approaches into requirements generation pipelines may further enhance the interpretability and reliability of LLM outputs, representing an interesting avenue for future research.

While our study does not propose a fundamentally new evaluation paradigm or algorithmic technique, its contribution is methodological and domain specific. We adapt, operationalize, and systematically validate the LLM-as-a-Judge paradigm for the Requirements Engineering context by reformulating ISO/IEC/IEEE 29148:2018 intrinsic quality attributes into machine operational criteria, designing a fully reproducible evaluation pipeline, and establishing the first large scale benchmark of requirement generation directly from minimal OSS artifacts. This represents a novel empirical framework for RE, enabling future studies to build upon standardized prompts, datasets, and evaluation procedures specifically tailored to the challenges of generating and assessing requirements under minimal-context conditions.

5 Threats to validity

We discuss potential threats to the validity of our findings according to established categories [51]. *Construct validity* concerns the relationship between the theoretical constructs and their operational measurements. (1) *Measuring requirement quality:* We measured quality using *Unambiguity*, *Verifiability*, and *Singularity*, derived from ISO/IEC/IEEE 29148:2018. While justified in the Method section based on their assessability in isolation for OSS data, these metrics represent only a subset of the characteristics defined by the standard (e.g., omitting Feasibility, Completeness, Correctness). Our findings reflect quality primarily through

this specific lens. Moreover, as our study evaluates only the intrinsic linguistic formulation of individual requirement statements, we do not attempt to judge contextual ISO attributes such as Necessity, Completeness, or Feasibility, which inherently require domain knowledge or stakeholder input. Evaluating such properties from issue titles would produce unreliable or misleading conclusions; (2) *LLM-as-a-Judge as quality proxy:* The evaluation relied on an LLM (Qwen QwQ-32b) as a judge. Although this approach offers scalability and consistency, and previous work suggests a high agreement with humans [20, 55], the interpretation of the quality criteria by the LLM judge might differ subtly from human experts. However, recent domain-specific evidence strengthens the validity of this proxy: Casciani et al. [7] reported that LLM judges evaluating formal requirement models achieved near-perfect alignment with human experts (Pearson correlation $r > 0.97$), demonstrating that LLMs can reliably assess semantic soundness in RE contexts. Furthermore, as noted in the survey by Li et al. [20], traditional automated metrics like BLEU fail to capture subtle attributes in open-ended generation, making LLM-based evaluation the current state-of-the-art for scalable assessment of intrinsic quality. We mitigated inherent biases by providing clear definitions based on the standard and capturing the judge's rationale.

To empirically validate the LLM judge, we conducted a human validation study on a stratified sample of 50 requirements (Sect. 4.6). The results confirmed statistically significant correlations between the LLM-judge scores and aggregate human judgment for all three attributes ($p < 0.05$), with within-± 1 agreement ranging from 72 to 96%. While a full human evaluation of all 900 artifacts remains infeasible, this targeted validation on a representative sample, covering all LLM, prompt, and score-bin combinations provides empirical evidence that the LLM-as-a-Judge approach can serve as a practically useful and reasonably aligned approximation for intrinsic linguistic quality assessment. For transparency and future verification, we provide the complete set of 900 generated requirements and 900 LLM-judge rationales in our open replication package, enabling independent human review if desired; and (3) *Representing feature requests by titles:* We used only GitHub issue titles as input for requirement generation. Titles may not capture the full context or nuance present in the issue body or comments. This simplification, made for consistency and feasibility, means our results reflect the LLMs' ability to generate requirements from concise, potentially incomplete, initial statements rather than fully detailed requests.

A further limitation concerns the fact that Qwen QwQ-32B is a general purpose model rather than a domain-specific requirements engineering expert. Since our evaluation does not seek to judge domain correctness, feasibility, or stakeholder alignment, but only the intrinsic linguistic properties of each requirement, this limitation does not undermine the validity of the assessment. However, we acknowledge that evaluating domain adequacy or contextual appropriateness would require human experts or domain tuned evaluators, and lies outside the scope of the present work.

Internal validity addresses factors that could influence the observed outcomes other than the experimental manipulations. (1) *Confounding effect of input quality:* The inherent clarity, complexity, and domain specificity of the input issue titles varied significantly, influencing the quality of the generated requirements independently of the LLM or prompt used. While we analyzed the repository effect as a proxy, controlling precisely for input quality across all 150 diverse titles was infeasible; and (2) *LLM Non-Determinism:* Although we used fixed seeds and temperature settings, inherent non-determinism in LLM APIs or underlying optimizations could potentially introduce slight variations. While we observed practical reproducibility during our experiments, we cannot guarantee identical outputs upon exact replication.

External validity concerns the generalizability of our findings. (1) *Limited LLM selection:* We evaluated only two LLMs (`o3-mini` and `DeepSeek R1`). Findings regarding performance and sensitivity to prompt styles may not generalize directly to other models (e.g., larger OpenAI models, Claude, Llama variants) with different architectures, training data, or fine-tuning; (2) *Specific prompt techniques:* We tested three specific prompt styles (Naive, Few-shot, Expert Identity). The relative effectiveness might differ with other techniques (e.g., Chain-of-Thought, complex templates, retrieval-augmented generation); (3) *Task and input specificity:* Our study focused on generating single requirement statements from issue titles. The results may not apply to generating requirements from different inputs (e.g., user stories, full descriptions, meeting notes) or to other RE tasks (e.g., classification, tracing, refinement). Consequently, the generated outputs are not intended to be directly adopted as actionable project specifications, but rather as draft textual artifacts that require further human refinement and integration into a project specific requirements baseline; and (4) *Context of OSS repositories:* The data originated from five specific, active open-source repositories. Practices for writing feature requests and the nature of requirements may differ in other OSS projects, smaller projects, different domains (e.g., safety-critical systems), or commercial/closed-source development environments.

Conclusion validity and reliability relates to the ability to draw correct conclusions and the study's repeatability. (1) *Reliability of LLM judge:* Beyond construct validity concerns, the judge's consistency in applying the scoring criteria across 900 requirements could be questioned. Potential

fatigue effects are minimized compared to humans, but internal model consistency isn't guaranteed. We used structured JSON output to ensure ratings were captured systematically. Moreover, the human validation study (Sect. 4.6) provides empirical evidence that the judge's scores are consistent with aggregate human assessment, with Spearman correlations significant at $p < 0.05$ for all three attributes. (2) *Statistical Power and Tests:* We used appropriate non-parametric tests for ordinal data. While the sample size (N = 900) provides reasonable power, the possibility of Type II errors (missing small true effects) exists, as with any statistical analysis; (3) *Data Extraction and Processing:* Potential errors in using the GitHub API or in processing scripts could affect the dataset. Standard API usage and script verification were employed to minimize this risk; and (4) *Reproducibility:* As noted under Internal Validity, LLM non-determinism poses a challenge. Furthermore, the rapid evolution of LLMs means the specific model versions used (`o3-mini`, `DeepSeek R1`, `Qwen QwQ-32b`) might be updated or deprecated, potentially affecting future replication attempts. We clearly documented the models and parameters used. To further support reproducibility, all datasets, evaluation scripts, and figures are openly available in our replication package on Zenodo (https://doi.org/10.5281/zenodo.15003691). This repository ensures full traceability of the experimental workflow and enables independent validation of all analyses presented in this study.

6 Future work

Future research should extend this study by systematically evaluating additional LLMs and advanced prompting strategies (e.g., Chain-of-Thought and retrieval-augmented generation) to better understand how reasoning mechanisms and external knowledge sources influence the intrinsic quality of generated requirements. In particular, future work should investigate how LLMs handle vague or underspecified issue titles, including techniques for disambiguation, requirement decomposition, and controlled enrichment of missing contextual information.

Another promising direction is the integration of domain knowledge into the generation process, for example through retrieval-based methods or structured prompt templates, to improve both verifiability and contextual adequacy while preserving requirement atomicity.

In addition, a direct comparison between LLM-based evaluation and traditional assessment approaches (e.g., expert review or rule-based evaluators) would require richer contextual artifacts and a dedicated experimental design. Such a comparison would help clarify under which conditions automated evaluation can complement or reliably approximate human judgment in requirements engineering tasks.

Finally, while our human validation study provides evidence that LLM-based evaluation aligns with human judgment for intrinsic linguistic quality, future work should investigate contextual ISO/IEC/IEEE 29148 attributes such as correctness, completeness, necessity, and feasibility. Evaluating these aspects would require domain knowledge and stakeholder input, representing a complementary line of research that could further strengthen the practical applicability of LLM-assisted requirements engineering.

7 Conclusion

This study investigated the potential of Large Language Models (LLMs) to automatically generate software requirements from concise feature requests (issue titles) found in open-source software repositories, with a particular focus on how different prompt engineering strategies influence the quality of the output. Using an LLM-as-a-Judge approach to evaluate 900 generated requirements based on ISO/IEC/IEEE 29148:2018 quality attributes (*Unambiguity*, *Verifiability*, *Singularity*), we found that LLMs can produce requirements that are often rated highly on these metrics. However, significant variability exists, heavily influenced by the clarity of the input issue title. Our findings confirm that prompt engineering is crucial, significantly affecting requirement quality, but its impact is notably model-dependent. For instance, while Few-shot prompting often improved Singularity for both tested LLMs, the Expert Identity prompt yielded mixed results, enhancing Verifiability for one model (`DeepSeek R1`) but degrading Singularity for both, revealing potential trade-offs between detail and focus. Neither LLM consistently outperformed the other across all metrics, although `o3-mini` showed a slight advantage in generating more singular requirements.

The primary contribution of this work is the empirical demonstration of these nuanced interactions between specific LLMs, prompt styles, and requirement quality attributes in the context of generating requirements from OSS issue titles, evaluated scalably via LLM-as-a-Judge. A targeted human validation study with five evaluators confirmed that the LLM judge constitutes a reliable proxy for human expert judgment, with statistically significant correlations across all three quality attributes. Our results suggest that while LLMs are promising tools for automating initial requirement drafting, they function best as assistants rather than fully autonomous solutions. Achieving consistently high-quality, usable requirements necessitates careful consideration of the specific LLM, tailored prompt design, and, crucially, human oversight for refinement and validation.

Although our approach builds on existing LLM-based evaluation concepts, its innovation lies in tailoring these methods to the RE domain, operationalizing ISO-based linguistic criteria for automated assessment, and establishing a reproducible empirical foundation for future research on LLM-assisted requirements generation and evaluation.

Author contributions All authors contributed equally to this work and reviewed the final manuscript.

Funding The Article Processing Charge (APC) for the publication of this research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) (ROR identifier: 00x0ma614).

Data availability The dataset, images, and scripts that support the findings of this study are openly available in Zenodo at https://doi.org/10.5281/zenodo.15003691

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Alhoshan W, Ferrari A, Zhao L (2023) Zero-shot learning for requirements classification: an exploratory study. Inf Softw Technol 159:107202. https://doi.org/10.48550/arXiv.2302.04723
2. Almonte JT, Boominathan SA, Nascimento N (2025) Automated non-functional requirements generation in software engineering with large language models: a comparative study. arXiv:2503.15248
3. Alves APS, Kalinowski M, Mendez D et al (2025) Industrial practices of requirements engineering for ml-enabled systems in Brazil: an extended analysis. J Softw Eng Res Dev 13:(2):13-208–13:223. https://doi.org/10.5753/jserd.2025.5689 (https://journals-sol.sbc.org.br/index.php/jserd/article/view/5689)
4. Arora C, Grundy J, Abdelrazek M (2023) Advancing requirements engineering through generative AI: assessing the role of llms. CoRR 2310:13976. https://doi.org/10.48550/arXiv.2310.13976
5. Attanayaka B, Nawinna DP, Manathunga K et al (2022) Success factors of requirement elicitation in the field of software engineering. In: 2022 4th International Conference on Advancements in Computing (ICAC). https://doi.org/10.1109/ICAC57685.2022.10025166
6. Brown TB, Mann B, Ryder N, et al (2020) Language models are few-shot learners. https://doi.org/10.48550/arXiv.2005.14165. arXiv:abs/2005.14165
7. Casciani A, Giacomo GD, Marrella A, et al (2025) A requirements engineering-driven methodology for planning domain generation via llms with invariant-based refinement. In: Proceedings of the ICAPS 2025 Workshop on Planning in the Era of LLMs (LM4Plan), Melbourne, Australia. https://icaps25.icaps-conference.org/program/workshops/lm4plan/
8. Cohen J (1968) Weighted kappa: nominal scale agreement provision for scaled disagreement or partial credit. Psychol Bull 70(4):213–220. https://doi.org/10.1037/h0026256
9. Cruzes DS, Dybå T (2011) Recommended steps for thematic synthesis in software engineering. Int Symp Empir Softw Eng Measur (ESEM) IEEE. https://doi.org/10.1109/ESEM.2011.36
10. DeepSeek-AI, Guo D, Yang D, et al (2025) Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv:2501.12948
11. Delima R, Mustofa K, Sari AK (2023) Automatic requirements engineering: activities, methods, tools, and domains—a systematic literature review. Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi). https://doi.org/10.29207/resti.v7i3.4924
12. Devlin J, Chang M, Lee K, et al (2019) BERT: pre-training of deep bidirectional transformers for language understanding. In: Burstein J, Doran C, Solorio T (eds) Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers). Association for Computational Linguistics, pp 4171–4186. https://doi.org/10.18653/V1/N19-1423
13. Dong Q, Li L, Dai D, et al (2024) A survey on in-context learning. arXiv:2301.00234
14. Dunn OJ (1964) Multiple comparisons using rank sums. Technometrics 6(3):241–252. https://doi.org/10.1080/00401706.1964.10490181 (http://www.tandfonline.com/doi/abs/10.1080/00401706.1964.10490181)
15. Fan W, Ding Y, Ning L, et al (2024) A survey on rag meeting llms: towards retrieval-augmented large language models. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. Association for Computing Machinery, New York, NY, USA, KDD '24, p 6491–6501. https://doi.org/10.1145/3637528.3671470
16. Femmer H, Vogelsang A (2019) Requirements quality is quality in use. IEEE Softw 36(3):83–91. https://doi.org/10.1109/MS.2018.110161823
17. Franch X, Palomares C, Quer C et al (2023) The state-of-practice in requirements specification: an extended interview study at 12 companies. Require Eng 28:377–409. https://doi.org/10.1007/s00766-023-00399-7
18. Frattini J, Montgomery L, Fischbach J et al (2023) Requirements quality research: a harmonized theory, evaluation, and roadmap. Requir Eng 28(4):507–520. https://doi.org/10.1007/S00766-023-00405-Y
19. Großer K, Ahmadian AS, Rukavitsyna M et al (2024) Benchmarking requirement template systems: comparing appropriateness, usability, and expressiveness. Requir Eng 29(4):481–522. https://doi.org/10.1007/S00766-024-00427-0
20. Gu J, Jiang X, Shi Z, et al (2025) A survey on llm-as-a-judge. arXiv:2411.15594
21. He J, Shi J, Zhuo TY, et al. (2025) From code to courtroom: Llms as the new software judges. CoRR. https://doi.org/10.48550/ARXIV.2503.02246
22. Holtzman A, Buys J, Du L, et al (2020) The curious case of neural text degeneration. In: International Conference on Learning Representations. https://openreview.net/forum?id=rygGQyrFvH

23. Hou X, Zhao Y, Liu Y, et al (2023) Large language models for software engineering: a systematic literature review. https://doi.org/10.48550/arXiv.2308.10620. arxiv:2308.10620
24. Ibtasham MS, Bashir S, Abbas M, et al (2025) Reqrag: enhancing software release management through retrieval-augmented llms: An industrial study. In: Hess A, Susi A (eds) Requirements Engineering: Foundation for Software Quality - 31st International Working Conference, REFSQ 2025, Barcelona, Spain, April 7-10, 2025, Proceedings, Lecture Notes in Computer Science, vol 15588. Springer, pp. 277–292. https://doi.org/10.1007/978-3-031-88531-0_20
25. IEEE (2018) ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering. ISO/IEC/IEEE 29148:2018(E), pp. 1–104. https://doi.org/10.1109/IEEESTD.2018.8559686
26. Jin D, Jin Z, Chen X, et al. (2024) Mare: multi-agents collaboration framework for requirements engineering. https://doi.org/10.48550/arXiv.2405.03256
27. Krippendorff K (2011) Computing krippendorff's alpha-reliability. Departmental Papers (ASC), pp. 1–10. https://repository.upenn.edu/asc_papers/43
28. Krishna M, Gaur B, Verma A, et al (2024) Using llms in software requirements specifications: An empirical evaluation. In: Liebel G, Hadar I, Spoletini P (eds) 32nd IEEE International Requirements Engineering Conference, RE 2024, Reykjavik, Iceland, June 24-28, 2024. IEEE, pp. 475–483. https://doi.org/10.1109/RE59067.2024.00056
29. Kruskal WH, Wallis WA (1952) Use of ranks in one-criterion variance analysis. J Am Stat Assoc 47(260):583–621. https://doi.org/10.1080/01621459.1952.10483441 (http://www.tandfonline.com/doi/abs/10.1080/01621459.1952.10483441)
30. Li D, Jiang B, Huang L, et al (2025) From generation to judgment: opportunities and challenges of llm-as-a-judge. arXiv:2411.16594
31. Lian X, Wang S, Zou H et al (2025) Incorporating verification standards for security requirements generation from functional specifications. Proc ACM Softw Eng (FSE). https://doi.org/10.1145/3729347
32. Lian X, Wu J, Gao X, et al (2025) Vision to specification: Automating the transition from conceptual features to functional requirements. https://doi.org/10.48550/ARXIV.2505.12262
33. Lim S, Henriksson A, Zdravkovic J (2021) Data-driven requirements elicitation: a systematic literature review. SN Comput Sci. https://doi.org/10.1007/s42979-020-00416-4
34. Liu P, Yuan W, Fu J et al (2023) Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. ACM Comput Surv. https://doi.org/10.1145/3560815
35. Lu Y, Bartolo M, Moore A, et al (2021) Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. https://doi.org/10.18653/v1/2022.acl-long.556. arXiv:abs/2104.08786
36. Lubos S, Felfernig A, Tran TNT, et al (2024) Leveraging llms for the quality assurance of software requirements. In: Liebel G, Hadar I, Spoletini P (eds) 32nd IEEE International Requirements Engineering Conference, RE 2024, Reykjavik, Iceland, June 24-28, 2024. IEEE, pp. 389–397. https://doi.org/10.1109/RE59067.2024.00046
37. Mann HB, Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. Ann Math Stat 18(1):50–60. https://doi.org/10.1214/aoms/1177730491 (http://projecteuclid.org/euclid.aoms/1177730491)
38. Marques N, Silva RR, Bernardino J (2024) Using chatgpt in software requirements engineering: a comprehensive review. Fut Internet 16(6):1–21. https://doi.org/10.3390/fi16060180
39. Mehraj A, Zhang Z, Systä K (2024) A tertiary study on ai for requirements engineering. In: International Working Conference on Requirements Engineering: Foundation for Software Quality, REFSQ 2024. Springer, pp. 159–177
40. Mesquita R, Silva GRS, Canedo ED (2023) On the experiences of practitioners with requirements elicitation techniques. Proc XXXVII Braz Symp Softw Eng. https://doi.org/10.1145/3613372.3613410
41. Qwen Team (2025) QwQ-32B: embracing the power of reinforcement learning. https://qwenlm.github.io/blog/qwq-32b/
42. Ronanki K, Daniel BC, Horkoff J, et al (2023) Requirements engineering using generative ai: Prompts and prompting patterns. https://doi.org/10.48550/arXiv.2311.03832. arXiv:abs/2311.03832
43. Spearman C (1904) The proof and measurement of association between two things. Am J Psychol 15(1):72. https://doi.org/10.2307/1412159 (https://www.jstor.org/stable/1412159?origin=crossref)
44. Tizard J, Devine P, Wang H et al (2023) A software requirements ecosystem: Linking forum, issue tracker, and faqs for requirements management. IEEE Trans Software Eng 49(4):2381–2393. https://doi.org/10.1109/TSE.2022.3219458
45. Umar MA, Lano K (2024) Advances in automated support for requirements engineering: a systematic literature review. Requir Eng 29(2):177–207. https://doi.org/10.1007/S00766-023-00411-0
46. Umm-e-Habiba HM, Bogner J et al (2024) How mature is requirements engineering for ai-based systems? A systematic mapping study on practices, challenges, and future research directions. Requir Eng 29(4):567–600. https://doi.org/10.1007/S00766-024-00432-3
47. Wei J, Bosma M, Zhao V, et al (2022) Finetuned language models are zero-shot learners. In: International Conference on Learning Representations, pp. 1–46. https://doi.org/10.48550/arXiv.2109.01652
48. Wei J, Wang X, Schuurmans D, et al (2022) Chain of thought prompting elicits reasoning in large language models. arxiv:2201.11903
49. White C, Dooley S, Roberts M, et al (2024) Livebench: a challenging, contamination-free llm benchmark. arXiv:2406.19314
50. White J, Fu Q, Hays S, et al. (2023) A prompt pattern catalog to enhance prompt engineering with chatgpt. https://doi.org/10.48550/arXiv.2302.11382
51. Wohlin C, Runeson P, Höst M et al (2012) Experimentation in software engineering. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-662-69306-3
52. Xu B, Yang A, Lin J, et al (2023) Expertprompting: instructing large language models to be distinguished experts, pp. 1–8. https://doi.org/10.48550/arXiv.2305.14688. arXiv:abs/2305.14688
53. Yang A, Yang B, Zhang B, et al (2024) Qwen2.5 technical report. arXiv preprint arXiv:2412.15115
54. Zhao WX, Zhou K, Li J, et al (2023) A survey of large language models, pp. 1–140. https://doi.org/10.48550/arXiv.2303.18223. arxiv:2303.18223
55. Zheng L, Chiang WL, Sheng Y, et al (2023) Judging llm-as-a-judge with mt-bench and chatbot arena. arXiv:2306.05685

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.